06 多頁面設計

黃彬華編撰

- Bottom Sheet 頁面
- Share Sheet 頁面
- 導覽式頁面 - 建立
- 導覽式頁面 - 定義路徑
- 導覽式頁面 - 路徑控制
- 導覽式頁面 - 資料傳遞
- 導覽式頁面 - 資料存取
- 頁籤式頁面
- Composable 生命週期
- Composable 畫面的生命週期

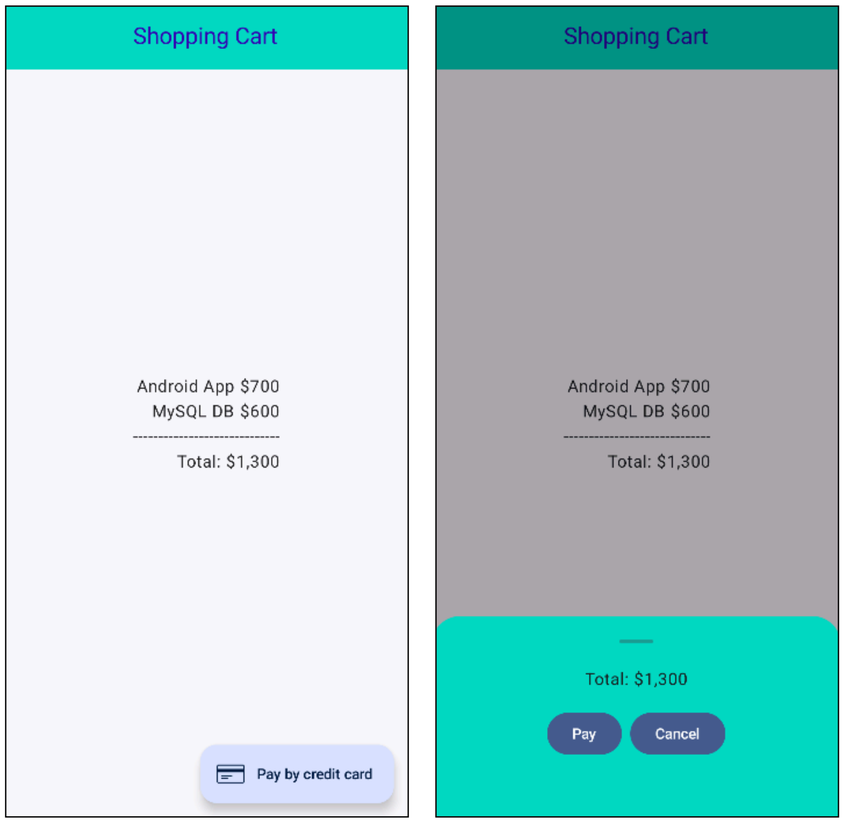
Bottom Sheet 頁面

黃彬華編撰

- 如果需要一個讓使用者互動的簡單頁面，而且顯示內容不多，尺寸不需要太大，可使用 bottom sheet
  - 開啟 bottom sheet 頁面後，使用者可以透過由上向下滑動方式關閉該頁面並回到前頁
  - 可用於支付頁面或版權宣告頁面
- 注意
  - 開啟 bottom sheet 後，前頁面仍舊可視，但會被灰色半透明罩住
  - **預覽無此效果**，需要使用模擬器或實機

範例 BottomSheetDemo

黃彬華編撰



- 第一頁點擊右下角浮動按鈕會從下方跳出 bottom sheet 頁面
- bottom sheet 頁面點擊 Pay 或 Cancel 按鈕都會回到前頁，但第一頁的付款狀態訊息會不同
- 注意：需要使用模擬器或實機測試

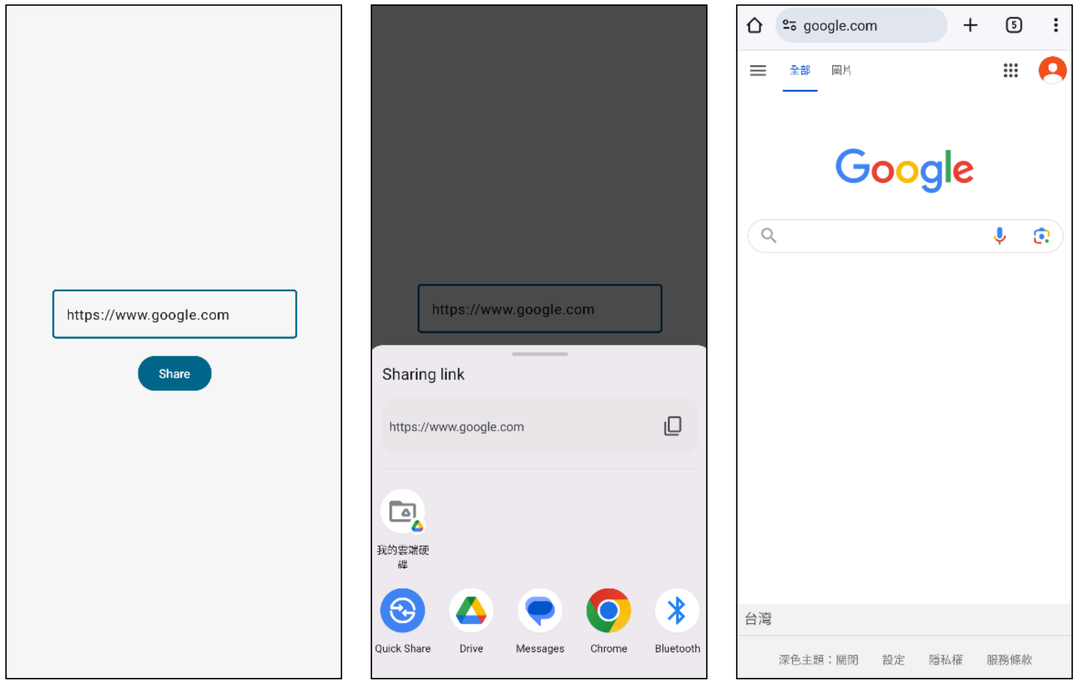
Share Sheet 頁面

黃彬華編撰

- 要讓使用者快速分享資訊到指定的 App，可以使用 share sheet 功能
  - 建立 ACTION_SEND Intent 並設定欲分享的資訊 (例如：網址)
  - 建立 ACTION_CHOOSER Intent 並包裝上述 Intent
  - 呼叫 startActivity() 開啟 share sheet 讓使用者可挑選欲分享的 App
- 參看 Send simple data to other apps
- 注意
  - 需要使用模擬器或實機測試，才能開啟 share sheet

範例 ShareSheetDemo

黃彬華編撰



- 第一頁輸入網址後點擊 Share 按鈕會跳出 share sheet 頁面，讓使用者可挑選欲分享的 App
- 使用者點選 Chrome App 就會將網址分享給該 App 並開啟對應網頁
- 注意：需要使用模擬器或實機測試

導覽式頁面 - 建立

黃彬華編撰

- 一個 App 往往有多個頁面，最常見的頁面切換方式就是導覽式 (navigation)
- 先建立頁面容器：使用 Composable 函式建立一個帶有 Scaffold 的頁面容器

```
@Composable
fun Main() {
  Scaffold() {
    …
  }
}
```

- 再建立各頁面：使用 Composable 函式建立其他頁面

```
@Composable
fun Screen01() {
  …
}
```

導覽式頁面 - 定義路徑

黃彬華編撰

- 前述頁面容器內 Scaffold 的內容使用 NavHost 定義各頁面的路徑 (route)，與該路徑所代表的頁面

```
NavHost() {
 composable(route = "Screen01") {
  Screen01()
 }
}
```

- 要切換到指定頁面需呼叫 navController.navigate() 搭配路徑

```
navController.navigate("Screen01") // 切換到上述 "Screen01" 路徑所代表的頁面 - Screen01
```

導覽式頁面 - 路徑控制

黃彬華編撰

- 導覽所經過頁面會使用 back stack 儲存
- 當導覽頁面切換方式不再只是下一頁、前一頁，而是可以直接去或回到指定頁面，NavHostController 物件就需要搭配下列函式功能來控制導覽路徑
  - 去指定頁面：navController.navigate("Screen02")
  - 回前頁：navigateUp() 與 popBackStack() 都可回前頁，但差別在於是否從其他 app 來此 app 首頁：
    - navigateUp()：可以回到來源 app，較適合用於左上角的 "Up" 按鈕
    - popBackStack()：只能單純回到前頁，而無法回到來源 app
  - 回指定頁面 (例如首頁)：navController.popBackStack("Screen01", false)
    - inclusive: false 代表不要從 back stack 移除當下頁面

導覽式頁面 - 資料傳遞

黃彬華編撰

- 導覽式頁面切換時往往需要將資料帶至下一頁，可透過參數方式傳遞
- 定義路徑時加上參數 - 使用 { } 框住參數名稱；並可透過 NavBackStackEntry 取值

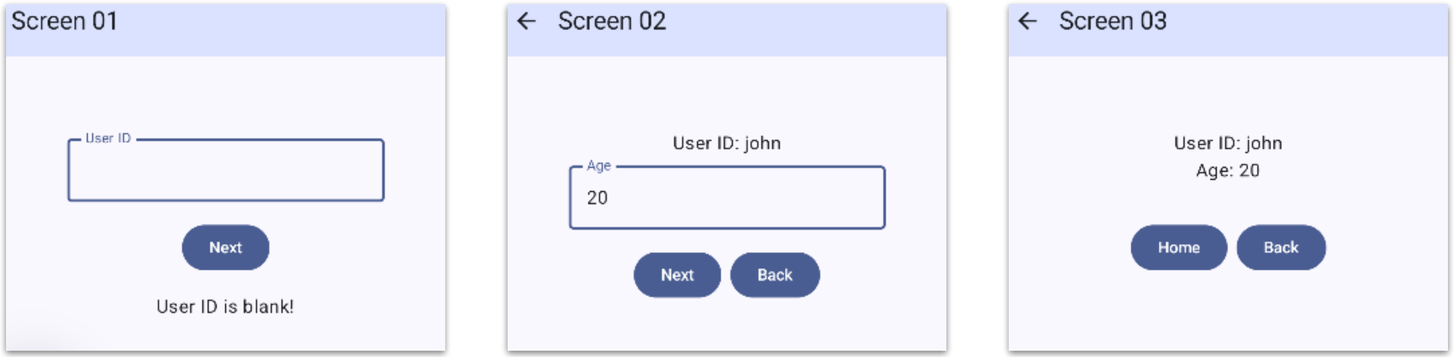
```
NavHost() {
  composable(route = "Screen02/{uid}") { backStackEntry ->
     Screen02(uid = backStackEntry.arguments?.getString("uid") ?: "")
  }
}
```

- 要切換到指定頁面時可加上欲傳遞的資料

```
navController.navigate("Screen02/John") // 切換到 Screen02 並將 John 傳給上述 uid 參數
```

範例 MultiScreensDemo

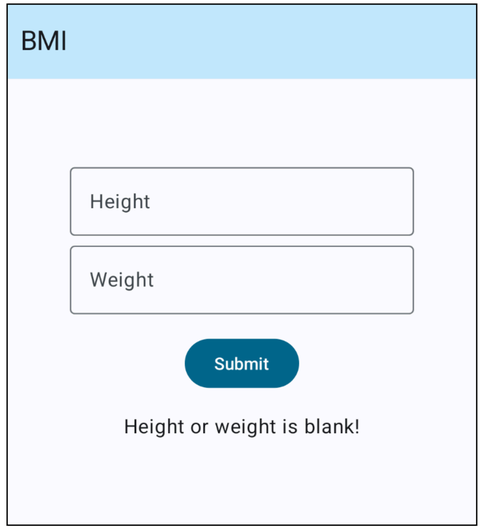
黃彬華編撰

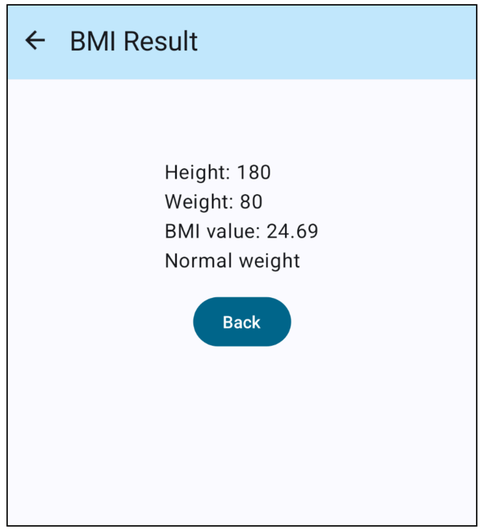


- Screen 01 頁面未輸入 user ID 就點擊 Next 按鈕會在下方跳出錯誤訊息；輸入後點擊 Next 按鈕會將 user ID 資料帶到 Screen 02 頁面
- Screen 02 頁面會呈現 user ID 資料
  - 點擊 Next 按鈕會將 user ID 與輸入的 age 資料帶到 Screen 03 頁面
  - 點擊 Back 按鈕會回到 Screen 01 頁面
- Screen 03 頁面會呈現 user ID 與 age 資料
  - 點擊 Home 按鈕會回到 Screen 01 頁面

Ex 6.1

黃彬華編撰





- 第 1 頁輸入身高、體重後點擊 Submit 按鈕會將資料帶到第 2 頁
  - 如果資料輸入錯誤會顯示錯誤訊息
- 第 2 頁會呈現下列資訊，點擊左上角箭頭或下方 Back 按鈕會回到前頁
  - 身高、體重
  - BMI 值
    - BMI = 體重(公斤) / 身高(公尺)的平方
  - 依照 BMI 值對照表顯示對應訊息
    - Underweight: BMI < 18.5
    - Normal weight: 18.5 <= BMI < 25
    - Overweight: 25 <= BMI < 30
    - Obese: BMI >= 30

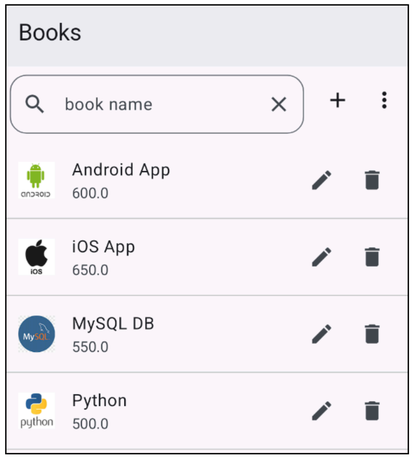
導覽式頁面 - 資料存取

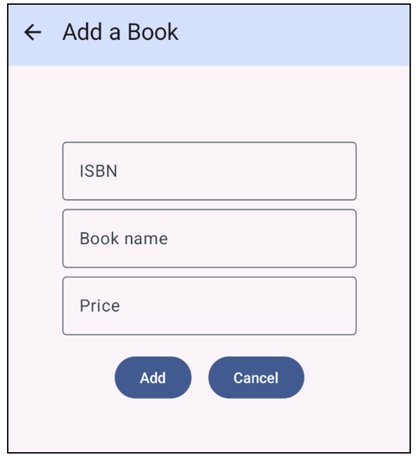
黃彬華編撰

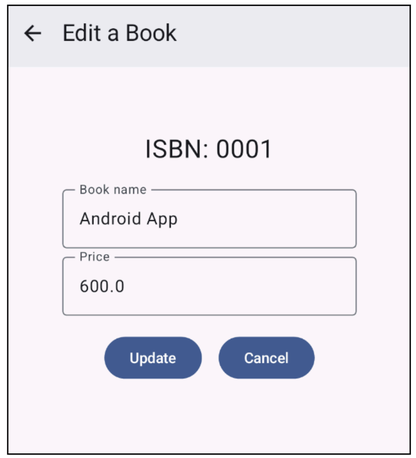
- 如果需要在生命週期 (lifecycle) 內持續存取該資料，可以搭配使用 MutableStateFlow 與 ViewModel
- 使用情境：購物型 App
  - 有許多頁面皆可購物，使用者購買商品會存放在購物車內；因為有多頁都需要存取購物車資料，所以建議搭配 MutableStateFlow 來記錄購物車狀態；然後透過 ViewModel 對購物車做增刪改查
  - 不建議使用前述以參數方式傳遞資料，因為資料僅能傳遞到下一頁，而無法處理跨多頁存取資料的狀況

範例 BooksCRUDDemo

黃彬華編撰



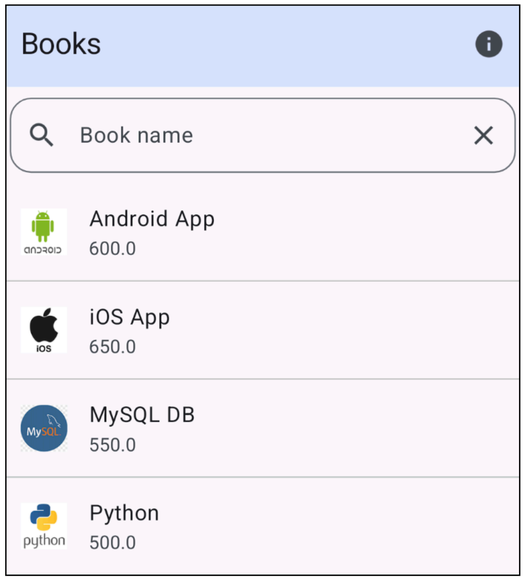


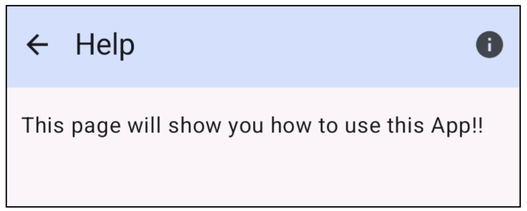


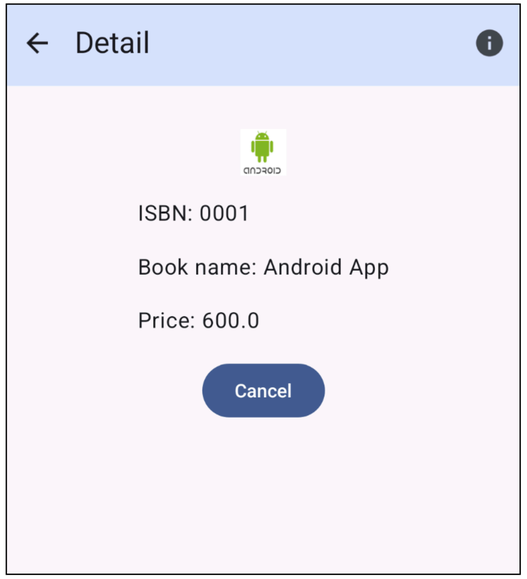
- 可關鍵字搜尋書籍資料
- 點擊新增按鈕會切換至「Add a Book」頁面，讓使用者可以新增一本書
- 點擊編輯按鈕會切換至「Edit a Book」頁面，讓使用者可以修改該書資料
- 點擊刪除按鈕會刪除該本書

Ex 6.2

黃彬華編撰





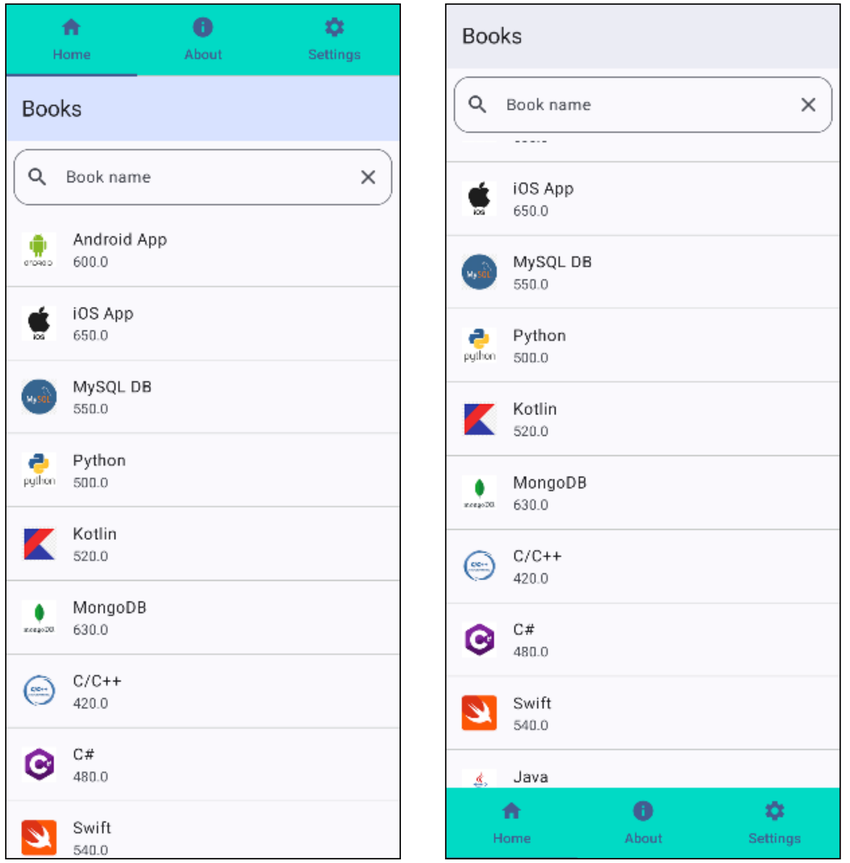


- 點擊 Books 頁面右上方 "i" 圖示會開啟 "Help" 頁面
- 點選 Books 頁面的任一項目後會開啟明細頁面顯示詳細資訊
  - 點擊左上角箭頭或下方 Cancel 按鈕會回到 Books 頁面

頁籤式頁面

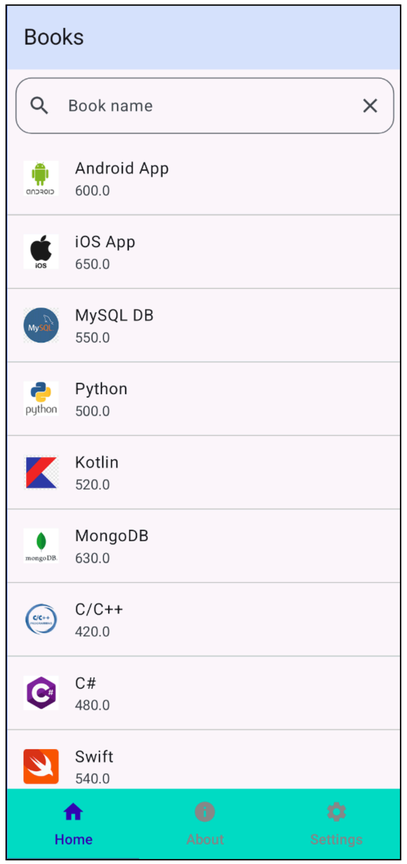
黃彬華編撰

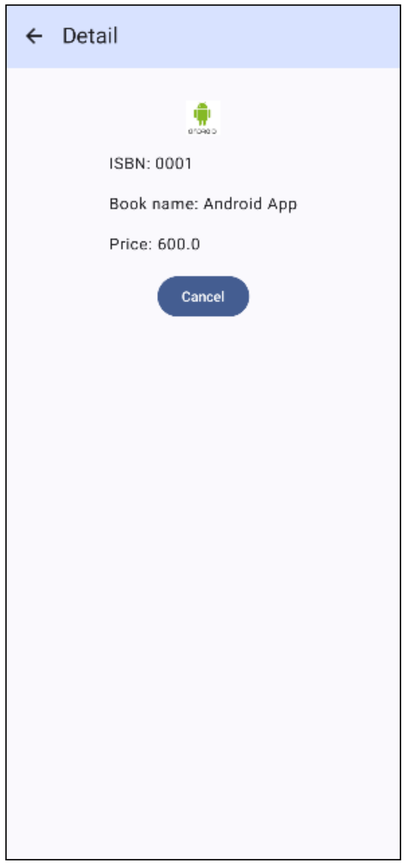
- 建立頁籤式 (tab) 頁面，需要使用 TabRow 功能
  - TabRow 可視需要放在頁面內容的上方或下方
  - 改用 ScrollableTabRow，當頁籤數量過多時可以滑動
- 建立步驟
  - 建立 list 存放頁籤標題
  - 建立 TabRow 並完成各個 Tab 選項的標題與圖示設定
  - 最後依照使用者選取的頁籤來指定要開啟的頁面
- TabRow 的隱藏與顯示
  - 如果進入指定頁面，需要將 TabRow 隱藏或顯示，可以在 ViewModel 宣告 tabVisibility 屬性，以便於控制 TabRow 的隱藏與顯示



範例 TabsDemo

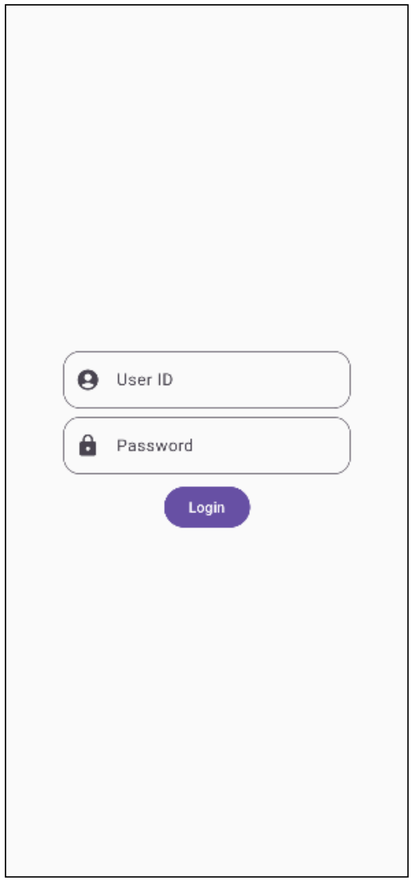
黃彬華編撰

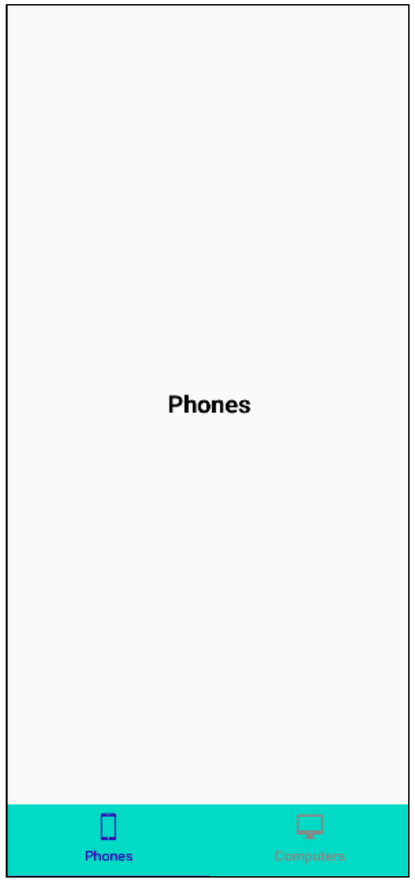




- 點選不同的頁籤會開啟不同頁面
- 進入書籍明細頁面時，會將 TabRow 隱藏

Ex 6.3



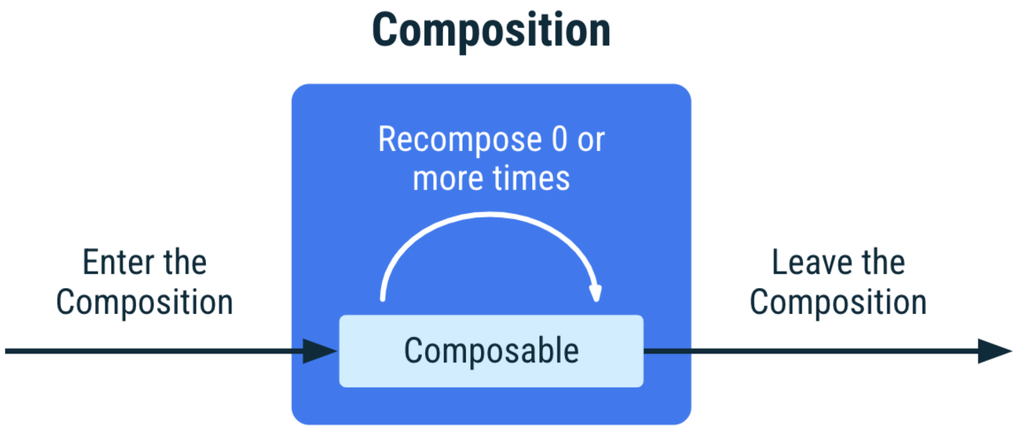


- 第一頁為登入畫面，點擊 Login 按鈕會進入第二頁
- 第二頁有 Phones 與 Computers 兩個頁籤，點擊後會呈現對應畫面

Composable 生命週期

黃彬華編撰

- composition 是建立、更新和維護 UI 的過程
- composable 生命週期 由下列事件構成
  - 第一次進入 composition (又稱為 initial composition)
  - recompose 0 至多次 (又稱為 recomposition)
  - 離開 composition

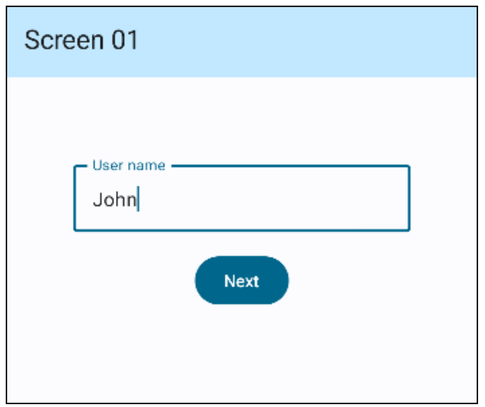


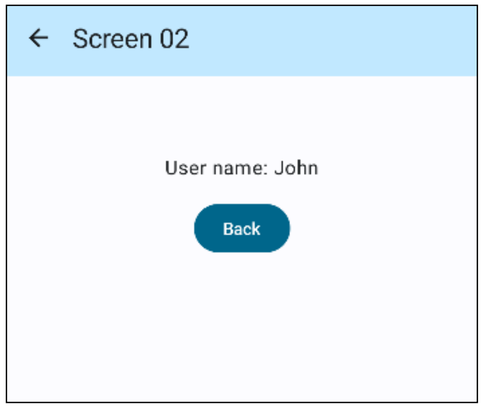
Composable 畫面的生命週期

黃彬華編撰

- 每個 composable 都有 lifecycle owner，可透過 LocalLifeCycleOwner.current 取得
  - 在 DisposableEffect 內利用 lifecycle owner 來加入 observer 以監控 composable 畫面的生命週期，這樣可以對使用者動作做更細膩的回應
- composable 畫面 (view) 的生命週期事件
  - ON_CREATE：第一次建立畫面時
  - ON_START：畫面準備要呈現時
  - ON_RESUME：畫面將要與使用者互動時
  - ON_PAUSE：畫面將要暫停，使用者無法操作時
  - ON_STOP：當畫面停止呈現時
  - ON_DESTROY：準備結束，畫面將要被移除時

範例 LifecycleDemo





- 在兩頁間切換並觀察 composable 與其畫面的生命週期變化